



Sécurité des systèmes

Cours 4 – Exercices - Shellcoding & memory permissions

Exercise 1

2



- **Let's write a shellcode.. that gives a shell**
 - Look at the exemples in exercices/loader
 - Write a shellcode that executes `/bin/sh`
 - By calling the appropriate syscall (`execve`)
 - Test your shellcode with the loader

Exercise 2

3



■ **Let's do cours2/exo2 (reuse your exploit code of exo1)**

- Goal : Get a remote shell on the existing TCP connection (this is more stealthy & closer to a real-life attack)
- 1 - Modify your shellcode to use the socket as stdin, stdout, stderr
 - `dup2(sock, 0) ; dup2(sock, 1) ; dup2(sock, 2) ; execve(...)`
- 2 - Resolve the address of `mprotect()`
- 3 - Run your shellcode !
 - Place the shellcode in the `username[]` buffer
 - Call `mprotect()` in the process to make the buffer executable
 - Set the return address of `mprotect` to the username buffer

Exercise 2

4



- **You should get a shell**
 - The remote process will be replaced by /bin/sh
 - The inputs/output of /bin/sh will be your socket
- **You can send commands on the socket & read the result**
 - Implement a small python shell to run commands

Exercise 2

5



■ **Reminder on how to resolve mprotect()**

- mprotect is in the libc
- How to find the base address of the libc ?
 - You know how to call functions in the target process
 - Let's force the process to disclose the address of the libc !
 - call write(4, GOT@plt, 32)
 - 4 is the socket number of your connection
 - You will obtain pointers to functions of the libc
 - Use a known pointer to compute the base address of the libc
 - Then add the right offset to find mprotect()



■ List of syscalls

- `/usr/include/asm/unistd_32.h`



<https://www.linkedin.com/company/synacktiv>

<https://twitter.com/synacktiv>

Nos publications sur : <https://synacktiv.com>